# ✈️ TRAVEL

## Plataforma Web Integral de Gestión y Reservas de Paquetes Turísticos

**DOCUMENTO DE ESPECIFICACIÓN DE SOFTWARE Y ARQUITECTURA DE SISTEMAS**

**Carrera:** Tecnicatura Superior en Desarrollo de Software

**Materia:** Arquitectura y Diseño de Interfaces Gráficas

**Profesor:** Prof. Juan Caballero

**Curso / Año Lectivo:** 3ro 2da — Año 2026

**Integrantes del Equipo y Distribución de Roles Técnicos:**

- **Alvarado, Abel:** Backend Lead & Lógica de Negocio (Arquitectura Flask RESTful, Blueprints, Modelos y Control de Overbooking).
- **Alvarado, Melanie:** Frontend & UI/UX Lead (Diseño React 19, Tailwind CSS, navegación fluida y experiencia de reserva).
- **Villegas, Mayra:** DevOps, Base de Datos & QA Lead (Contenedorización Docker Compose, Migraciones y Suite de Pruebas Automatizadas).

## 1. Resumen Ejecutivo

El presente proyecto integrador tiene como propósito diseñar, estructurar e implementar una solución web integral para la digitalización y gestión comercial de una agencia de turismo receptivo y nacional en la República Argentina. La plataforma **TRAVEL** unifica la oferta de hospedajes de alta gama, itinerarios exclusivos y paquetes todo incluido en una sola interfaz reactiva, garantizando un flujo de reserva seguro, validado en tiempo real y gestionado por roles diferenciados (Cliente, Administrador Comercial y Administrador General).

## 2. Modelo de Negocio Actual

Las agencias de turismo tradicionales suelen operar bajo esquemas descentralizados basados en planillas estáticas de cálculo, consultas informales por mensajería y procesos manuales de control de cupos. Esta fragmentación operativa genera inconsistencias en la disponibilidad, sobreventas involuntarias de habitaciones y lentitud en la confirmación de itinerarios. **TRAVEL** responde a este problema centralizando el catálogo de destinos con sincronización de inventario, cálculo automatizado de tarifas con promociones estacionales y un canal formal de confirmación de solicitudes de viaje.

## 3. Objetivos del Sistema

**Objetivo General:** Desarrollar una plataforma web completa bajo arquitectura MVC desacoplada (Frontend reactivo SPA + Backend API RESTful) que automatice la publicación, comercialización y gestión de reservas de experiencias turísticas argentinas, asegurando interfaces estéticas, reactivas y confiables.

**Objetivos Específicos:**

- Centralizar y categorizar las experiencias turísticas (Solo Estadía, Viaje con Aventura y All Inclusive) con desglose de itinerarios y servicios.
- Brindar a los clientes un buscador reactivo con filtros combinados por provincia, tipo de experiencia y tope de presupuesto.
- Implementar un sistema de autenticación robusto basado en JSON Web Tokens (JWT) y cifrado criptográfico de contraseñas.
- Prevenir de forma algorítmica el solapamiento de fechas y sobreventas (Overbooking) en el motor de reservas.
- Proveer a la administración comercial y general tableros analíticos en tiempo real con métricas financieras y auditoría de personal.

## 4. Actores del Sistema

| Actor / Stakeholder | Descripción y Responsabilidades en el Sistema |
|---|---|
| **Cliente / Turista** | Usuario que explora el catálogo público, aplica filtros por provincia y presupuesto, crea su cuenta, inicia sesión y formaliza solicitudes de reserva accediendo a su historial personal en «Mis Reservas». |
| **Administrador Comercial** | Operador administrativo encargado de la gestión comercial del catálogo: alta/baja lógica de destinos, suspensión temporal de visibilidad en temporada baja, aplicación de descuentos promocionales (10%, 25%, 50% OFF) y actualización de estados operativos de reservas (Pendiente, Confirmada, Cancelada). |
| **Administrador General (SuperAdmin)** | Máxima autoridad técnica y directiva de la plataforma. Supervisa auditorías globales, monitorea ingresos acumulados mediante tableros analíticos con gráficos vectoriales nativos y administra las credenciales y roles del personal. |

## 5. Necesidades Planteadas (Matriz DINO)

La priorización de requerimientos se estructuró bajo la metodología DINO (Obligatorio, Importante, Deseable, No Implementable) orientada al alcance del Proyecto Integrador:

| N° | Necesidad Planteada | Clasificación | Justificación Técnica / Comercial |
|---|---|---|---|
| 1 | Autenticación segura con JWT y roles jerárquicos | **O (Obligatorio)** | Pilar de seguridad; restringe endpoints administrativos y asocia reservas al cliente. |
| 2 | Catálogo público reactivo de experiencias y hoteles de Argentina | **O (Obligatorio)** | Núcleo de comercialización; exhibe tarifas base, fotos, itinerarios y servicios. |
| 3 | Paneles de control diferenciados (Comercial vs General) | **O (Obligatorio)** | Asegura el principio de mínimo privilegio entre operaciones comerciales y auditoría directiva. |
| 4 | ABM integral de alojamientos y control de visibilidad | **O (Obligatorio)** | Permite pausar o reactivar destinos según temporadas sin eliminar datos históricos. |
| 5 | Filtros reactivos (por provincia, tipo de viaje y presupuesto) | **I (Importante)** | Optimiza sustancialmente la experiencia de usuario (UX) en la localización del hospedaje. |
| 6 | Motor de reservas con validación de fechas y cálculo de tarifas | **I (Importante)** | Calcula noches efectivas, descuentos porcentuales vigentes y tasas de servicio de forma exacta. |
| 7 | Prevención algorítmica de solapamiento de fechas (Anti-Overbooking) | **I (Importante)** | Rechaza con código HTTP 409 solicitudes sobre rangos previamente ocupados. |
| 8 | Tablero de Analítica Financiera con gráficos vectoriales SVG nativos | **I (Importante)** | Provee curvas mensuales de facturación y distribución de reservas por provincia en tiempo real. |
| 9 | Pasarela de pago online 100% bancarizada con tarjeta | **D (Deseable)** | En esta primera versión la reserva se formaliza con código y confirmación administrativa manual. |
| 10 | Asistente virtual conversacional con IA para soporte 24/7 | **N (No Impl.)** | Excede el alcance operativo y requerimientos de la cátedra de Interfaces Gráficas. |

## 6. Requerimientos Funcionales (RF)

| Código | Requerimiento Funcional | Detalle de Implementación en el Sistema |
|---|---|---|
| **RF 1** | Registro de usuarios seguro | Creación de cuentas con nombre, correo electrónico único y validación de contraseña mínima de 6 caracteres. |

| Código | Requerimiento Funcional | Detalle de Implementación en el Sistema |
|---|---|---|
| **RF 2** | Autenticación e inicio de sesión | Validación de credenciales con emisión de Bearer Token JWT para el consumo de la API REST. |
| **RF 3** | Cifrado criptográfico de contraseñas | Hasheo irreversible de claves mediante el algoritmo Werkzeug Scrypt antes de almacenar en base de datos. |
| **RF 4** | Catálogo público reactivo | Renderizado de tarjetas con precio por noche, calificación, insignias de descuento y galería de imágenes. |
| **RF 5** | Modal interactivo de detalle | Apertura de cajón lateral con carrusel fotográfico, lista de actividades del itinerario y comodidades incluidas. |
| **RF 6** | Filtros combinados de búsqueda | Filtrado simultáneo por provincia de Argentina, rango de precio máximo y categoría de experiencia turística. |
| **RF 7** | Generación de solicitudes de reserva | Registro de reserva vinculada al usuario autenticado, indicando fechas de entrada/salida y cantidad de huéspedes. |
| **RF 8** | Validación de coherencia temporal | Comprobación de que la fecha de check-in sea igual o posterior al día actual y que el check-out sea estrictamente posterior al check-in. |
| **RF 9** | Control algorítmico de Overbooking | El backend rechaza con estado 409 solicitudes cuyas fechas se crucen con reservas ya confirmadas o pendientes del mismo hotel. |
| **RF 10** | Historial de reservas del cliente | Vista exclusiva «Mis Reservas» con el costo total liquidado, estado actual (Pendiente, Confirmada, Cancelada) y detalles. |
| **RF 11** | Cancelación de reservas por el cliente | El turista puede dar de baja reservas pendientes desde su panel personal, liberando inmediatamente las fechas bloqueadas. |
| **RF 12** | Control de visibilidad del catálogo | El administrador comercial puede pausar destinos para ocultarlos temporalmente del catálogo público sin borrarlos de la BD. |
| **RF 13** | Gestión de promociones estacionales | Aplicación instantánea de descuentos (10%, 25%, 50% OFF) recalculando la tarifa neta mostrada a los visitantes. |
| **RF 14** | ABM completo de hospedajes | Endpoints REST para creación (POST), modificación (PUT) y eliminación definitiva (DELETE) de experiencias turísticas. |
| **RF 15** | Gestión administrativa de reservas | El personal operativo puede listar todas las reservas globales y cambiar su estado a «Confirmada» o «Cancelada». |
| **RF 16** | Administración de roles de usuario | El Administrador General puede promover o degradar roles (cliente, comercial, general) desde una tabla reactiva en vivo. |

| Código | Requerimiento Funcional | Detalle de Implementación en el Sistema |
|---|---|---|
| **RF 17** | Protección contra auto-degradación | El sistema impide que el único Administrador General del sistema se retire su propio rol, resguardando el acceso de auditoría. |
| **RF 18** | Métricas financieras en tiempo real | Cálculo dinámico de ingresos totales acumulados, tasa de conversión estimada y comparativa porcentual de facturación mensual. |
| **RF 19** | Gráficos analíticos vectoriales | Gráfico de barras SVG nativo para ingresos mes a mes y gráfico de donut para la distribución de demanda por provincia. |
| **RF 20** | Persistencia de sesión segura | El frontend detecta la presencia del token JWT en localStorage al refrescar la pantalla y restaura el perfil y vista adecuada. |

## 7. Requerimientos No Funcionales (RNF)

| Código | Requerimiento No Funcional | Criterio de Aceptación y Estándar Técnico |
|---|---|---|
| **RNF 1** | Diseño Responsive Adaptable | La interfaz se adapta fluidamente a teléfonos móviles, tablets y escritorios mediante grillas CSS y Tailwind v4. |
| **RNF 2** | Tiempo de respuesta eficiente | Tiempos de respuesta inferiores a 1.5 segundos en la carga inicial y menos de 100 ms en el filtrado reactivo del catálogo. |
| **RNF 3** | Arquitectura Desacoplada | Separación estricta entre capa cliente (SPA en React 19) y capa servidora (API RESTful en Flask con Blueprints). |
| **RNF 4** | Compatibilidad Cruzada | Funcionamiento y renderizado visual idéntico garantizado en Google Chrome, Microsoft Edge, Mozilla Firefox y Safari. |
| **RNF 5** | Micro-animaciones ergonómicas | Transiciones suaves con Framer Motion en apertura de modales, tarjetas y retroalimentación interactiva (toasts). |
| **RNF 6** | Resiliencia en persistencia | Soporte dual de base de datos relacional: conexión directa con MySQL 8.0 y fallback transparente a SQLite para desarrollo sin fallos. |
| **RNF 7** | Calidad y Cobertura de Pruebas | Suite de 9 pruebas automatizadas unitarias ejecutadas sobre base de datos en memoria (100% de aprobación en ~1 segundo). |
| **RNF 8** | Reproducibilidad con Docker | Configuración completa en Docker Compose que orquesta contenedores para MySQL, Flask API y Frontend Web con un solo comando. |
| **RNF 9** | Seguridad en Comunicaciones | Headers de seguridad CORS configurados y protección de endpoints mediante decorador personalizado `@role_required`. |
| **RNF 10** | Integración Continua (CI/CD) | Workflow automatizado con GitHub Actions para correr la suite de pruebas en cada push o pull request al repositorio. |
| **RNF 11** | Mantenibilidad de Código | Estructura modular dividida en controladores (`routes/`), lógica de dominio (`services/`) y utilidades (`utils/`). |

## 8. Historias de Usuario (HU)

| ID | Rol | Historia de Usuario (Quiero / Para) | Criterio de Aceptación |
|---|---|---|---|
| **HU 01** | Visitante | Quiero registrarme con mi correo y contraseña para acceder a la reserva de hospedajes. | El formulario valida email válido, contraseña $\ge$ 6 caracteres y emite token JWT inmediato. |
| **HU 02** | Turista | Quiero filtrar por provincia y presupuesto máximo para encontrar un viaje acorde a mis posibilidades. | El catálogo se refresca al instante sin recargar la página conforme se desplaza el slider de precio. |

| ID | Rol | Historia de Usuario (Quiero / Para) | Criterio de Aceptación |
|---|---|---|---|
| **HU 03** | Turista | Quiero ver el itinerario completo y comodidades de un hotel para conocer los servicios incluidos. | Al pulsar la tarjeta se despliega el cajón lateral con fotos, lista de actividades y amenidades. |
| **HU 04** | Cliente logueado | Quiero solicitar una reserva indicando check-in y check-out para asegurar mi alojamiento. | El sistema valida que no haya sobreventa de fechas, calcula el precio total con tasas y guarda en estado «Pendiente». |
| **HU 05** | Cliente | Quiero cancelar una reserva que ya no usaré para liberar el cupo. | Botón «Cancelar Reserva» en «Mis Reservas» que actualiza el estado a «Cancelada» de inmediato. |
| **HU 06** | Admin Comercial | Quiero ocultar un hotel de la vista pública para gestionar temporada baja o mantenimiento. | Interruptor de visibilidad que modifica el campo booleano `activo` en la API REST. |
| **HU 07** | Admin Comercial | Quiero aplicar descuentos del 10%, 25% o 50% OFF para impulsar la venta de alojamientos destacados. | Selector que actualiza la tarifa base y muestra la insignia de oferta con precio tachado. |
| **HU 08** | Admin Comercial | Quiero aprobar reservas recibidas para formalizar la contratación de los clientes. | Listado con selector de estado para cambiar entre «Pendiente», «Confirmada» y «Cancelada». |
| **HU 09** | SuperAdmin | Quiero visualizar la facturación acumulada del año para monitorear el rendimiento del negocio. | Gráfico de barras SVG con los ingresos reales desglosados mes a mes y tarjeta de facturación total. |
| **HU 10** | SuperAdmin | Quiero modificar el rol de un colaborador para otorgar o remover permisos comerciales o de auditoría. | Tabla de personal con selector desplegable en vivo que actualiza el rol en la base de datos. |

## 9. Diagrama y Modelo Entidad-Relación (DER)

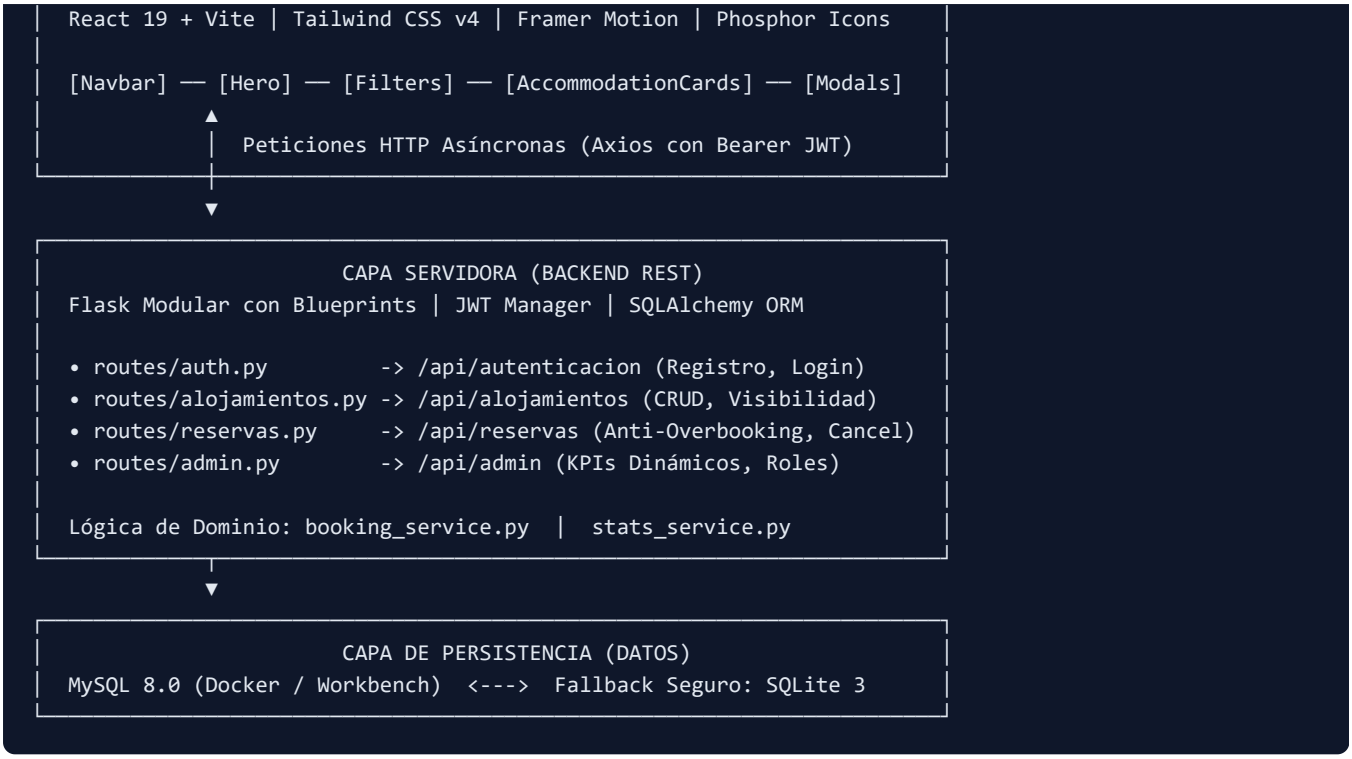
El diseño relacional fue optimizado bajo un **modelo integrado de experiencias turísticas**. En lugar de fragmentar artificialmente las consultas de destinos, hoteles y actividades en múltiples llamadas lentas, la entidad `alojamiento` unifica la oferta principal y se vincula relacionalmente con sus itinerarios (`actividad`), amenidades (`servicio`) y contratos de viaje (`reserva`), resguardando la integridad referencial mediante cascada de borrado (CASCADE).

| Entidad | Clave Primaria (PK) | Claves Foráneas (FK) | Atributos Principales y Relaciones |
|---|---|---|---|
| **usuario** | `id` (Integer Autoincremental) | — | `nombre` (Varchar 100), `email` (Varchar 120, Unique, Index), `password_hash` (Varchar 256), `rol` ('cliente', 'comercial', 'general'), `fecha_creacion` (DateTime). Relación 1:N con `reserva`. |
| **alojamiento** | `id` (Varchar 50 - Slug semántico) | — | `titulo` (Varchar 150), `ubicacion` (Varchar 100), `provincia` (Varchar 100), `precio` (Int), `calificacion` (Float), `reviews_count` (Int), `imagenes_json` (Text), `tipo` ('estadia', 'viaje', 'all_inclusive'), `experience_label` (Varchar 100), `descripcion` (Text), `destacado` (Boolean), `activo` (Boolean), `descuento` (Int). Relación 1:N con `actividad`, `servicio` y `reserva`. |
| **actividad** | `id` (Integer Autoincremental) | `alojamiento_id` $ightarrow$ alojamiento.id (ON DELETE CASCADE) | `descripcion` (Varchar 255). Itinerario y actividades exclusivas del alojamiento. Relación N:1 con `alojamiento`. |
| **servicio** | `id` (Integer Autoincremental) | `alojamiento_id` $ightarrow$ alojamiento.id (ON DELETE CASCADE) | `nombre` (Varchar 150). Amenities incluidas (Piscina, Cava, Spa, Wifi, etc.). Relación N:1 con `alojamiento`. |
| **reserva** | `id` (Integer Autoincremental) | `usuario_id` $ightarrow$ usuario.id `alojamiento_id` $ightarrow$ alojamiento.id | `check_in` (Date), `check_out` (Date), `huespedes` (Int), `notes` (Text), `precio_total` (Int), `estado` ('pending', 'confirmed', 'cancelled'), `fecha_creacion` (DateTime). Relación N:1 con `usuario` y `alojamiento`. |

## 10. Diagrama de Arquitectura y Clases (MVC Desacoplado)

> **Patrón Arquitectónico Implementado: API-First Decoupled MVC**
>
> La aplicación adopta una arquitectura desacoplada donde el **Frontend (React SPA)** y el **Backend (Flask REST API)** operan de forma independiente comunicándose mediante contratos JSON sobre protocolo HTTP.




React 19 + Vite | Tailwind CSS v4 | Framer Motion | Phosphor Icons
[Navbar] ── [Hero] ── [Filters] ── [AccommodationCards] ── [Modals]
Peticiones HTTP Asíncronas (Axios con Bearer JWT)
CAPA SERVIDORA (BACKEND REST)
Flask Modular con Blueprints | JWT Manager | SQLAlchemy ORM
• routes/auth.py -> /api/autenticacion (Registro, Login)
• routes/alojamientos.py -> /api/alojamientos (CRUD, Visibilidad)
• routes/reservas.py -> /api/reservas (Anti-Overbooking, Cancel)
• routes/admin.py -> /api/admin (KPIs Dinámicos, Roles)
Lógica de Dominio: booking_service.py | stats_service.py
CAPA DE PERSISTENCIA (DATOS)
MySQL 8.0 (Docker / Workbench) <---> Fallback Seguro: SQLite 3

## 11. Mapeo de Modelos Backend (Flask) y Componentes Frontend (React)

| Modelo Backend (models.py) | Endpoints REST Consumidos | Componentes Visuales de la Interfaz |
|---|---|---|
| **Usuario** | `POST /api/autenticacion/registro`<br>`POST /api/autenticacion/ingreso`<br>`GET /api/autenticacion/perfil`<br>`PUT /api/admin/usuarios/<id>/rol` | `AuthModal.jsx` (Formularios interactivos con pestañas de login/registro).<br>`Navbar.jsx` (Identificador de usuario logueado y dropdown de cierre de sesión).<br>`AdminGeneralDashboard.jsx` (Tabla de gestión de roles de personal). |
| **Alojamiento** | `GET /api/alojamientos`<br>`GET /api/alojamientos/<id>`<br>`POST /api/alojamientos` (Crear)<br>`PUT /api/alojamientos/<id>/visibilidad`<br>`PUT /api/alojamientos/<id>/promocion` | `Hero.jsx` (Buscador rápido de destinos de Argentina).<br>`Filters.jsx` (Filtros por provincia, presupuesto y modalidad de viaje).<br>`AccommodationCard.jsx` (Tarjetas interactivas con insignias y fotos).<br>`AdminComercialDashboard.jsx` (Gestor de visibilidad y descuentos en vivo). |
| **Actividad & Servicio** | Servidos anidados dentro del payload serializado de `GET /api/alojamientos/<id>` | `DetailModal.jsx` (Cajón lateral emergente que lista el cronograma de actividades exclusivas y la iconografía de amenidades disponibles). |
| **Reserva** | `POST /api/reservas` (Crear con anti-overbooking)<br>`GET /api/reservas` (Mis reservas)<br>`PUT /api/reservas/<id>/cancelar`<br>`GET /api/admin/reservas`<br>`PUT /api/admin/reservas/<id>/estado` | `DetailModal.jsx` (Formulario con validación de fechas, cálculo automático de costo final por noches e input de notas).<br>`MisReservas.jsx` (Historial del cliente con estado, detalles y botón de cancelación).<br>`AdminGeneralDashboard.jsx` (Cálculo de métricas financieras y facturación mensual). |

## 12. Manual de Uso del Sistema

### 12.1 Flujo para Clientes y Turistas

1. **Exploración del Catálogo:** El usuario ingresa a la portada principal. En la barra de filtros puede seleccionar su destino preferido (Mendoza, Bariloche, Iguazú, Salta, Ushuaia, etc.), acotar su presupuesto máximo mediante el control deslizante o filtrar por tipo de viaje (Solo estadía, Viaje con Aventura o All Inclusive).
2. **Inspección del Hospedaje:** Al hacer clic en «Ver Experiencia» sobre una tarjeta, se abre el cajón lateral con carrusel fotográfico, especificación de actividades incluidas (catas de vino, trekkings, navegaciones) y comodidades (spa, wifi, piscina).
3. **Autenticación y Registro:** Si el usuario no ha iniciado sesión, presiona «Iniciar Sesión» en el menú superior o en la reserva. En un modal emergente puede ingresar con su cuenta o registrarse completando nombre, email y contraseña.

4. **Confirmación de Reserva:** Con sesión iniciada, el cliente selecciona fecha de entrada y salida, especifica número de pasajeros y notas de preferencia. Al confirmar, el sistema valida la disponibilidad temporal y calcula el precio final exacto con descuentos e impuestos.
5. **Gestión en «Mis Reservas»:** En el menú superior accede a su historial completo, donde puede verificar el estado de su viaje (Pendiente, Confirmada) o presionar «Cancelar Reserva» si decide no viajar.

### 12.2 Flujo para Administrador Comercial

1. **Acceso con Rol Comercial:** Al iniciar sesión con credenciales comerciales (ej. `comercial@travel.com`), el sistema redirecciona automáticamente a la vista de gestión comercial.
2. **Control de Visibilidad:** El operador cuenta con un interruptor instantáneo en cada ficha para activar o desactivar la visibilidad de los alojamientos según la temporada turística.
3. **Fijación de Promociones:** Mediante botones de acceso rápido, puede aplicar ofertas instantáneas del 10%, 25% o 50% OFF, recalculando las tarifas del catálogo de forma inmediata.
4. **Seguimiento de Reservas:** Consulta las solicitudes entrantes y aprueba formalmente las reservas recibidas.

### 12.3 Flujo para Administrador General (SuperAdmin)

1. **Tablero Financiero y Analítica:** Al ingresar con rol general (ej. `admin@travel.com`), accede al panel de control central. Visualiza tarjetas KPI con ingresos totales acumulados, total de reservas concretadas, tasa de conversión estimada y comparativa porcentual de facturación del mes actual versus el mes previo.
2. **Gráficos Vectoriales Nativos:** Examina el gráfico de barras SVG que detalla la facturación mensual acumulada durante el año y el gráfico circular de distribución de reservas por provincia argentina.
3. **Auditoría y Gestión de Roles:** En la sección inferior, consulta la nómina completa de usuarios del sistema y dispone de selectores desplegables para promover o degradar roles en vivo con validación de seguridad.

## 13. Enlaces al Código Fuente y Verificación de Calidad

El código fuente completo de la plataforma **TRAVEL** se encuentra disponible en los siguientes repositorios públicos de GitHub:

| Componente del Sistema | Tecnología Principal | Enlace al Repositorio en GitHub |
|---|---|---|
| **Frontend Web** | React 19, Vite, Tailwind CSS v4, Framer Motion | https://github.com/AbelEmir11/travel.git |
| **Backend API REST** | Python 3.13, Flask Modular (Blueprints), SQLAlchemy, JWT | https://github.com/AbelEmir11/travel-backend.git |

### Verificación Automatizada de Calidad (Testing Suite)

Para garantizar la fiabilidad del software exigida en una cátedra universitaria, el sistema cuenta con una batería de **9 pruebas unitarias automatizadas** que verifican los módulos críticos de autenticación, seguridad de contraseñas, cálculo de tarifas con descuentos y el algoritmo de prevención de sobreventas:

```
PS C:\proyectos_vscode  ravel   ravel-backend> .□env\Scripts\python -m unittest discover tests
.........
----------------------------------------------------------------------
Ran 9 tests in 1.058s

OK (Tests pasaron al 100% exitosamente sobre SQLite en memoria)
```

### Despliegue Contenedorizado con Docker Compose

La plataforma dispone de archivos `Dockerfile` y un manifiesto `docker-compose.yml` unificado que orquesta los tres servicios del sistema (Base de datos MySQL 8.0, Backend API en Flask y Frontend Web en Nginx) con un solo comando de terminal:

```
docker compose up --build
```

---